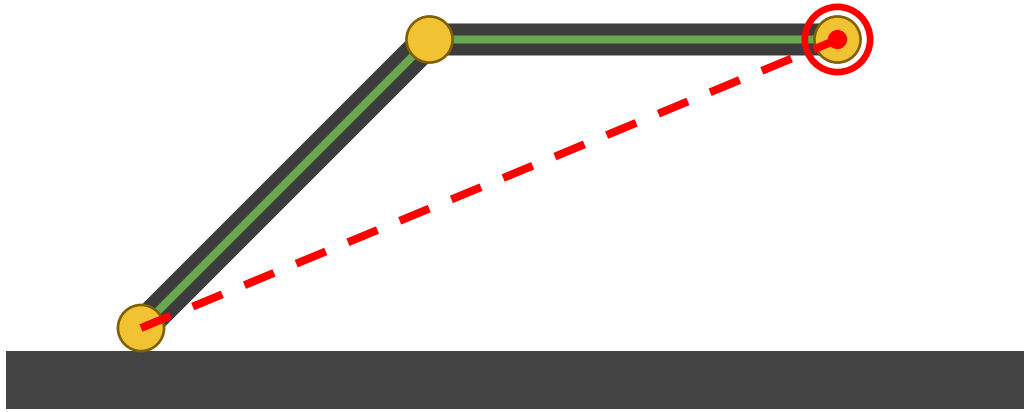
Code Implementation of Inverse Kinematics and Other Methods

Robotic Arm Course · Lesson 8

Day 1: Lesson 6’s triangles in Python
Day 2: Gradient descent in Python, and the Jacobian



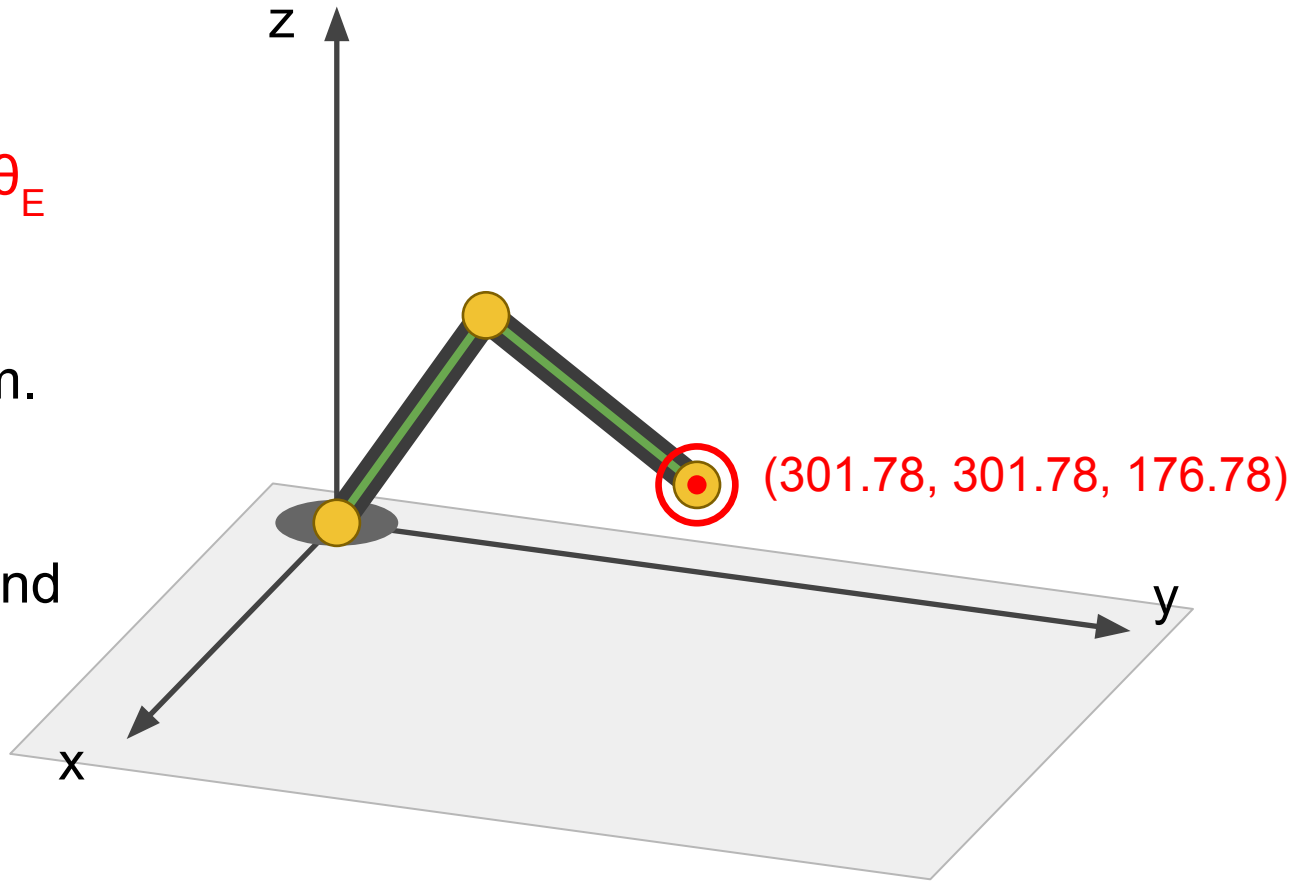
Where Lessons 5 to 7 left off:

Lesson 5: FK in code.
tip_3d(45, 45, -45) gives [301.78, 301.78, 176.78]

Lesson 6: triangles gave $\theta_B = 45°$, $\theta_S = 45°$, $\theta_E = -45°$, by hand.

Lesson 7: gradient descent got within 0.5 mm.
A computer took the 84 steps.

Today we write both IK methods in Python, and check every answer with FK from Lesson 5.



Setting up:

1. Make a folder with arm.py from Lesson 5 in it.
2. Open ik_starter.py and save a copy as ik.py in the same folder.
3. Run it. Nothing prints. If there is no error, it works.

We add to ik.py, one step at a time.

```
import math
from arm import L1, L2, show, flat_tip, tip_3d
```

The second line brings in Lesson 5's code: the link lengths, show(), and the FK functions flat_tip and tip_3d.

We use FK to check every IK answer.

From a number back to an angle:

Lesson 6 used $\cos^{-1}$ and $\tan^{-1}$. In Python they are math.acos and math.atan2.
Like sin and cos, they work in radians. math.degrees() changes radians back into degrees.

```
import math

print(math.acos(0))                         # 1.5707963267948966
print(math.degrees(math.acos(0)))           # 90.0
print(math.degrees(math.atan2(1, 1)))       # 45.0
```

math.acos(0) is 90° written in radians (π ÷ 2).
atan2(y, x) looks at the signs of both y and x, so it always points the right way (Lesson 6, Day 2).

Try it yourself #1: what does each line print?

```
print(round(math.degrees(math.acos(0.5)), 2))
print(round(math.degrees(math.atan2(1, -1)), 2))
print(math.acos(1.5))
```

Guess first. Then run them to check.

Hint: line 3 asks for the angle whose cosine is 1.5.

Try it yourself #1: solution

```
print(round(math.degrees(math.acos(0.5)), 2))
print(round(math.degrees(math.atan2(1, -1)), 2))
print(math.acos(1.5))
```

```
60.0
135.0
ValueError: math domain error
```

cos 60° = 0.5. The direction (−1, 1) is up and to the left: 135°.

No angle has a cosine bigger than 1, so Python stops with an error. In Lesson 6 that meant the target is out of reach.

Step 1: the law of cosines

```
def law_of_cosines(a, b, c):
    cos_c = (a ** 2 + b ** 2 - c ** 2) / (2 * a * b)
    cos_c = max(-1, min(1, cos_c))
    return math.degrees(math.acos(cos_c))
```

Test it:

```
print(round(law_of_cosines(250, 250, 300), 2))
```

```
73.74
```

The angle opposite side c, from Lesson 6:

cos C = (a² + b² − c²) ÷ (2ab)

Line 3 keeps cos_c between −1 and 1. Rounding can give 1.0000001, which would crash acos.

73.74° is Lesson 6's worked example.

Try it yourself #2: run law_of_cosines

Find the angle C for triangles with these sides:

1. 250, 250, 250
2. 250, 250, 400

Compare with Lesson 6, Try it yourself #1.

Try it yourself #2: solution

Find the angle C for triangles with these sides:
1. 250, 250, 250
2. 250, 250, 400

Compare with Lesson 6, Try it yourself #1.

```
print(round(law_of_cosines(250, 250, 250), 2))
print(round(law_of_cosines(250, 250, 400), 2))
```

```
60.0
106.26
```

All three sides equal gives 60°. In (2) the cosine is negative, so the angle is bigger than 90°.

Step 2: the elbow

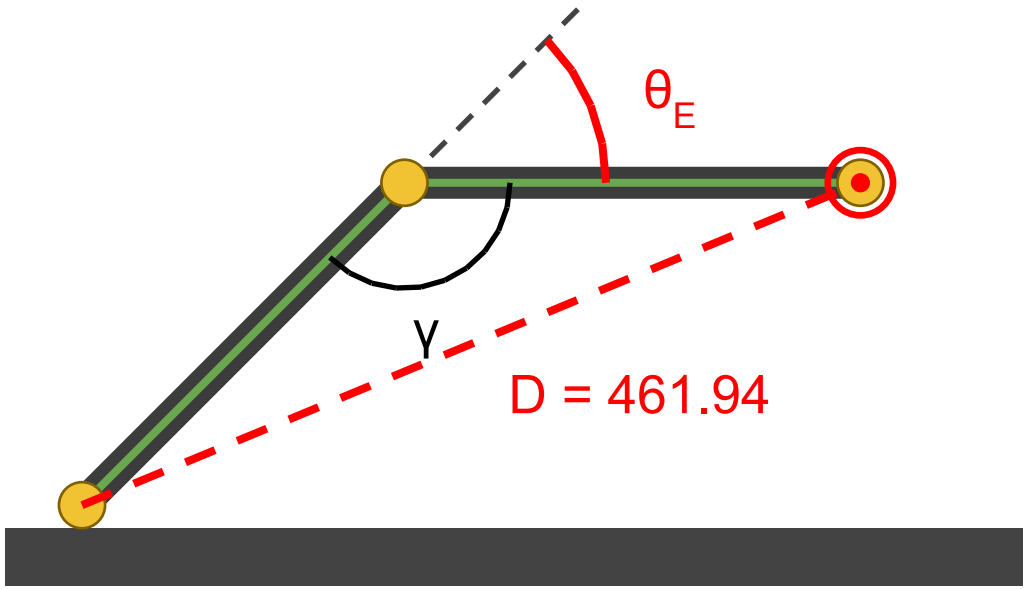
```
def elbow(D):
    gamma = law_of_cosines(L1, L2, D)
    return -(180 - gamma)
```

Test it:

```
print(round(elbow(461.94), 2))
print(round(elbow(353.55), 2))
```

```
-45.0
-90.0
```

γ is the angle inside the triangle at the elbow. The elbow bends 180° − γ away from a straight arm.
The minus sign: elbow up, the forearm bends down, so θ_E is negative.



Step 3: the shoulder

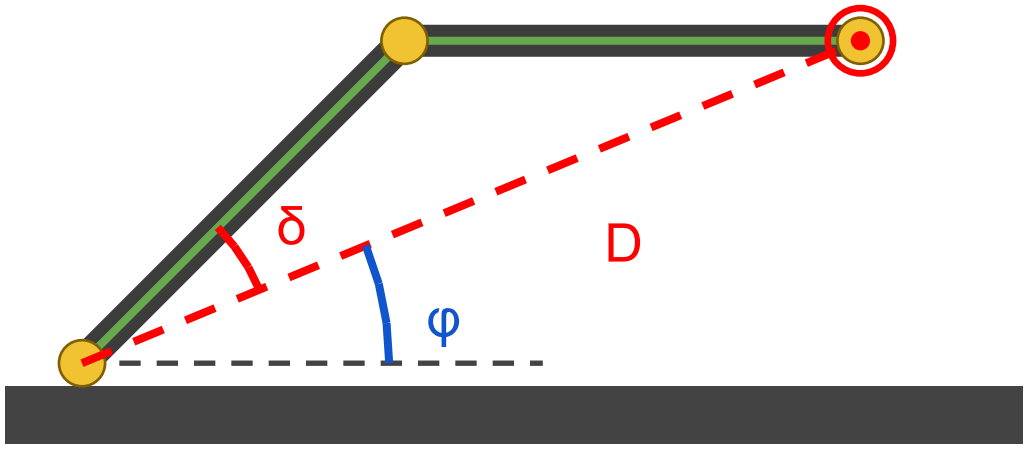
```
def shoulder(X, Y, D):
    phi = math.degrees(math.atan2(Y, X))
    delta = law_of_cosines(L1, D, L2)
    return phi + delta
```

Test it:

```
print(round(shoulder(426.78, 176.78, 461.94), 2))
```

```
45.0
```

φ is the tilt of the line D. δ is the angle between D and the upper arm, so the side opposite δ is L2. Elbow up: $\theta_S = \varphi + \delta$



Step 4: the flat arm

```
def ik_flat(X, Y):
    D = math.sqrt(X ** 2 + Y ** 2)
    if D > L1 + L2:
        raise ValueError("target is out of reach")
    return [shoulder(X, Y, D), elbow(D)]
```

Each line is one step of Lesson 6’s recipe for the flat arm.

D comes from the Pythagorean theorem.

If D is longer than both links together (500 mm), there is no triangle. raise stops the code with a message.

Then Steps 2 and 3 do the rest.

Run the worked example:

```
show(ik_flat(426.78, 176.78))
```

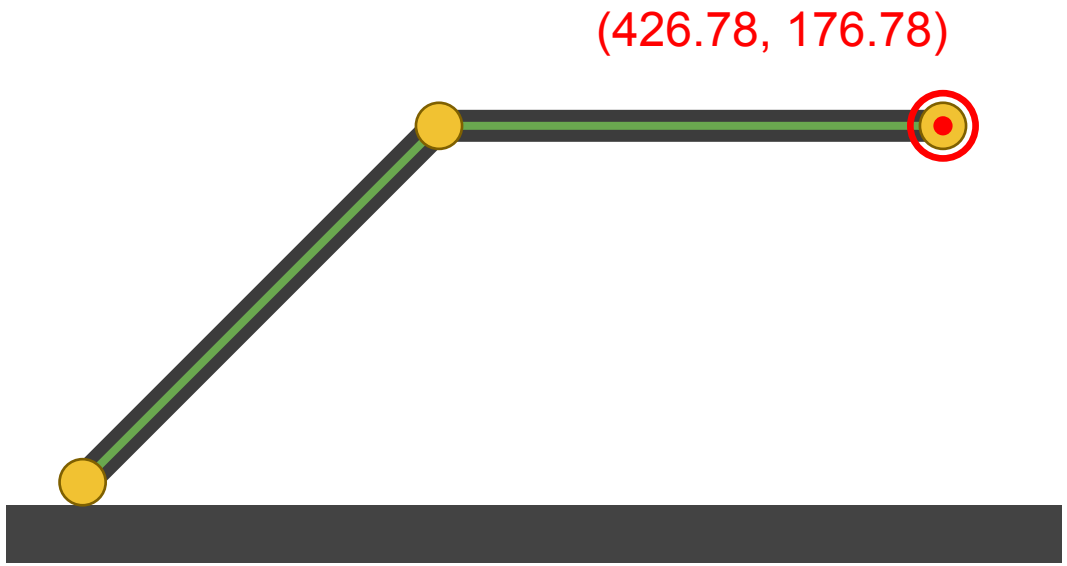
```
[45.0, -45.0]
```

Lesson 6 found $\theta_S = 45°$ and $\theta_E = -45°$.
Now check it with FK from Lesson 5:

```
show(flat_tip(45.0, -45.0))
```

```
[426.78, 176.78]
```

IK, then FK, brings us back to the target. Do this check every time.



Out of reach:

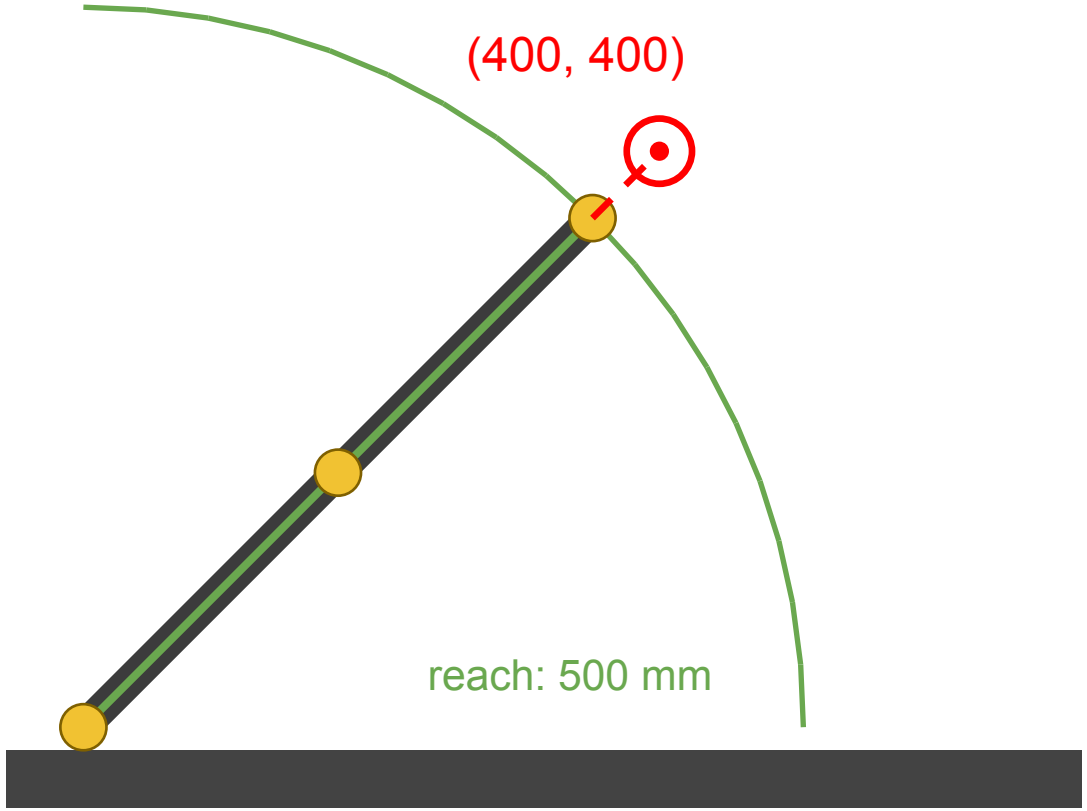
```
show(ik_flat(400, 400))
```

```
ValueError: target is out of reach
```

D = 565.69 mm, more than 500 mm. A robot must not move when there is no answer.
To print a message instead of stopping, use try and except:

```
try:
    show(ik_flat(400, 400))
except ValueError:
    print("out of reach")
```

```
out of reach
```



Try it yourself #3: run ik_flat

Find the angles for the targets:

1. (341.51, 341.51)
2. (250, 250)

Then check each answer with flat_tip. Compare with Lesson 6, Try it yourself #2 and #3.

Try it yourself #3: solution

Find the angles for the targets:
1. (341.51, 341.51)
2. (250, 250)

Then check each answer with flat_tip. Compare with Lesson 6, Try it yourself #2 and #3.

```
show(ik_flat(341.51, 341.51))
show(ik_flat(250, 250))
```

```
[60.0, -30.0]
[90.0, -90.0]
```

```
show(flat_tip(60, -30))
show(flat_tip(90, -90))
```

```
[341.51, 341.51]
[250.0, 250.0]
```

The same answers as Lesson 6, and FK puts both tips back on the targets. In (2) the upper arm points straight up and the forearm points straight across.

Step 5: the whole arm in 3D

```
def ik_3d(x, y, z):
    theta_b = math.degrees(math.atan2(y, x))
    R = math.sqrt(x ** 2 + y ** 2)
    theta_s, theta_e = ik_flat(R, z)
    return [theta_b, theta_s, theta_e]
```

Test it:

```
show(ik_3d(301.78, 301.78, 176.78))
show(tip_3d(45.0, 45.0, -45.0))
```

```
[45.0, 45.0, -45.0]
[301.78, 301.78, 176.78]
```

Lesson 6, Day 2:

1. atan2 aims the base.
2. R is the reach across the floor.
3. (R, z) is a flat-arm target, so ik_flat does the rest.

ik_flat already checks the reach, so ik_3d does not need its own check.

Try it yourself #4: run ik_3d

Find the angles for the targets:

1. (295.75, 170.75, 91.51)
2. (−324.76, 187.50, 216.51)

Check each answer with tip_3d. Compare with Lesson 6, Try it yourself #4 and #5.

Try it yourself #4: solution

Find the angles for the targets:
1. (295.75, 170.75, 91.51)
2. (−324.76, 187.50, 216.51)

Check each answer with tip_3d. Compare with Lesson 6, Try it yourself #4 and #5.

```
show(ik_3d(295.75, 170.75, 91.51))
show(ik_3d(-324.76, 187.50, 216.51))
```

```
[30.0, 60.0, -90.0]
[150.0, 60.0, -60.0]
```

tip_3d puts both tips back on the targets. In (2) atan2 gives 150°. Plain atan would give −30° (Lesson 6).

Day 1 recap:

law_of_cosines: an angle from three sides

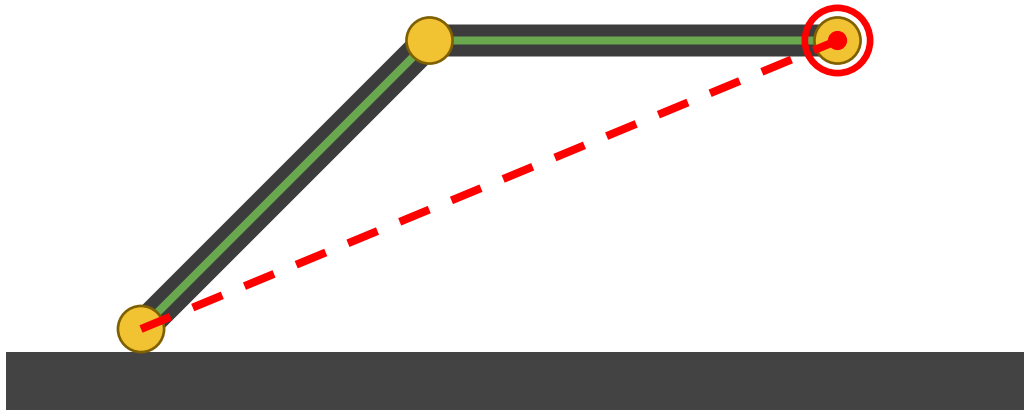
elbow and shoulder: Lesson 6's Steps 1 and 2

ik_flat: the flat arm, with a reach check

ik_3d: atan2 aims the base, then ik_flat

Always check IK with FK.

Tomorrow: Lesson 7's gradient descent in code.



Day 2: where Day 1 left off

1. What does ik_flat(400, 400) do?

2. Why does ik_3d use atan2 and not atan?

3. How do we check an IK answer?

Today Lesson 7’s loop becomes code: error, cost, slopes, and the loop itself.

Step 6: the error and the cost

```
def error(angles, target):
    degrees = [math.degrees(a) for a in angles]
    tip = tip_3d(*degrees)
    total = 0
    for i in range(3):
        total += (tip[i] - target[i]) ** 2
    return math.sqrt(total)

def cost(angles, target):
    return (error(angles, target) / L1) ** 2
```

Lesson 7's step rule uses radians, so the angles come in as radians.

Line 2 makes a new list with each angle in degrees, for tip_3d.

tip_3d(*degrees) puts the three numbers of the list in as three inputs.

The error is the distance in 3D. The cost is Lesson 7's C = (error ÷ 250)².

Test the error:

```
target = (301.78, 301.78, 176.78)
start = [math.radians(a) for a in (0, 45, -45)]
print(round(error(start, target), 2))
print(round(cost(start, target), 2))
```

```
326.64
1.71
```

The start pose (0°, 45°, −45°) puts the tip at (426.78, 0, 176.78). It is 326.64 mm from the target, the first row of Lesson 7's "Watching the error fall".

The cost is $(326.64 \div 250)^2 = 1.71$.

Step 7: the slopes

```
def slopes(angles, target, h=0.0001):
    result = []
    for i in range(3):
        up = list(angles)
        up[i] += h
        down = list(angles)
        down[i] -= h
        result.append((cost(up, target) - cost(down, target)) / (2 * h))
    return result
```

Test it:

```
show(slopes(start, target))
```

```
[-4.12, -0.71, 0.0]
```

Nudge one joint at a time, as in Lesson 7.

list(angles) makes a copy, so the other joints stay where they are.

Nudging up and down and dividing by 2h is fairer than nudging one side.

h = 0.0001 rad is a tiny turn.

Negative slopes: θ_B and θ_S must get bigger.

Step 8: the loop

```
def ik_gd(target, start=(0, 45, -45), eta=0.1,
          tol=0.5, max_steps=1000):
    angles = [math.radians(a) for a in start]
    steps = 0
    while steps < max_steps and error(angles, target) >= tol:
        g = slopes(angles, target)
        for i in range(3):
            angles[i] = angles[i] - eta * g[i]
        steps += 1
    degrees = [math.degrees(a) for a in angles]
    return degrees, steps, error(angles, target)
```

The defaults are Lesson 7's settings.

while repeats as long as there are steps left and the error is not small enough yet.

The update is Lesson 7's step rule: $\theta_{new} = \theta - \eta \times \text{slope}$

return sends back three things: the angles, the steps and the error.

Run the worked example:

```
angles, steps, err = ik_gd((301.78, 301.78, 176.78))
show(angles)
print(steps, round(err, 2))
show(tip_3d(*angles))
```

```
[45.0, 45.15, -45.29]
84 0.49
[301.45, 301.45, 176.63]
```

The same answer the computer gave in Lesson 7: 84 steps, and the tip is 0.49 mm from the target. Lesson 6's exact answer is (45°, 45°, −45°).

angles, steps, err = ... takes the three things that ik_gd returns.

Try it yourself #5: change the learning rate

Run the worked example with eta=0.02, eta=0.2 and eta=0.3. How many steps does each take? Guess first.

```
angles, steps, err = ik_gd((301.78, 301.78, 176.78), eta=0.2)
```

Try it yourself #5: solution

Run the worked example with eta=0.02, eta=0.2 and eta=0.3. How many steps does each take? Guess first.

```
angles, steps, err = ik_gd((301.78, 301.78, 176.78), eta=0.2)
```

η	steps	error at the end
0.02	364	under 0.5 mm
0.1	84	under 0.5 mm
0.2	52	under 0.5 mm
0.3	1000	288.01 mm

η = 0.3 is too big. The error bounces between 234.5 and 288.0 mm and never settles, as in Lesson 7. ik_gd stops at the step limit, so always look at the error.

Test both methods:

```
targets = [(301.78, 301.78, 176.78), (295.75, 170.75, 91.51),
           (-324.76, 187.50, 216.51), (400, 300, 200)]

for target in targets:
    print("target:", target)
    try:
        angles = ik_3d(*target)
        print("  trig:", end=" ")
        show(angles)
    except ValueError:
        print("  trig: out of reach")
    angles, steps, err = ik_gd(target)
    print("  gd:  ", end=" ")
    show(angles)
    print("        ", steps, "steps, error", round(err, 2), "mm")
```

check_ik.py runs both methods on targets from Lesson 6.

try and except catch the out of reach error, so the loop keeps going.

end=" " keeps show() on the same line.

Run it and compare the two answers for each target.

What the tests show:

Target	Trigonometry (Day 1)	Gradient descent (Day 2)
(301.78, 301.78, 176.78)	[45.0, 45.0, -45.0]	[45.0, 45.15, -45.29], 84 steps
(295.75, 170.75, 91.51)	[30.0, 60.0, -90.0]	[30.0, 59.91, -89.85], 79 steps
(−324.76, 187.50, 216.51)	[150.0, 60.0, -60.0]	[-30.0, 179.88, -59.78], 179 steps
(400, 300, 200)	out of reach	[36.87, 21.8, 0.0], 1000 steps, 38.52 mm

Rows 1 and 2: the same pose. Gradient descent is close, not exact.

Row 3: both are right. Gradient descent found the other valley: the base points away and the arm reaches back over the top (Lesson 7).

Row 4: trigonometry stops with an error. Gradient descent ends 38.52 mm short with no error. Always check the error.

The Jacobian: how the tip moves

How far does the tip move when one joint turns by 1°? Nudge that joint and run FK, as in Lesson 7.
At the pose (45°, 45°, −45°):

Joint turns 1°	**x (mm)**	**y (mm)**	**z (mm)**
θ_B	−5.27	5.27	0.00
θ_S	−2.18	−2.18	7.45
θ_E	0.00	0.00	4.36

This table is called the **Jacobian**. It says how each joint moves the tip.

θ_B swings the tip around the base. θ_E lifts it straight up, because the forearm points straight across in this pose (250 mm × 1° in radians = 4.36 mm).

A Jacobian method:

Gradient descent asks: which way is downhill? Then it takes a small step.
A Jacobian method asks: which turns of all three joints move the tip straight to the target? The table answers that.

```
python jacobian_preview.py
```

```
...
[45.0, 45.0, -45.0]
Jacobian: 5 steps, 0.01 mm
gradient descent: 84 steps, 0.49 mm
```

Target	Jacobian	GD
(301.78, 301.78, 176.78)	5	84
(295.75, 170.75, 91.51)	4	79
(−125, 216.51, 250)	11	73

You do not need to write this code. It limits each turn to 20° so it never jumps too far.
It has limits too: near a straight arm the table cannot push the tip farther out, and out of reach it also ends short.
Real robots add safety steps for that.

IK inside the arm’s own code:

```
def inverse_kinematics(x, y, z, t4=0.0):
    theta1 = math.atan2(y, x)
    r      = math.hypot(x, y)
    h      = z - L1
    d2     = r * r + h * h
    d      = math.sqrt(d2)

    if d > L2 + L3 or d < abs(L2 - L3) or d < 1e-9:
        return []

    cos_t3 = max(-1.0, min(1.0, (_L2_SQ + _L3_SQ - d2) / _2L2L3))
    theta3 = math.pi - math.acos(cos_t3)

    alpha    = math.atan2(h, r)
    cos_beta = max(-1.0, min(1.0, (_L2_SQ + d2 - _L3_SQ) / (_2L2 * d)))
    beta     = math.pi - (alpha + math.acos(cos_beta))
    theta2   = math.pi / 2 - beta + _ELBOW_OFF
```

From Code/waypoint.py, the arm’s control program.

Find the base angle, the reach, the reach check, the elbow (law of cosines) and the shoulder.

Here L2 and L3 are the links, 0.22 m each, and L1 is the shoulder height. It works in metres and radians, and measures the shoulder from straight up.

The steps are Lesson 6’s.

Three ways to do IK:

	Trigonometry	**Gradient descent**	**Jacobian**
Answer	exact	close (0.5 mm)	very close
Steps	one pass	50 to 1000	about 4 to 20
Needs	triangles for this arm	only FK	FK and the table
Out of reach	stops with an error	ends short, no error	ends short, no error
A new arm	new triangles	the same loop	the same loop

If the arm had six joints, which method would you use, and why?

Whichever you choose, check the answer with FK and look at the error.

What we learned:

1. Each step of Lesson 6's recipe became a small function: law_of_cosines, elbow, shoulder, ik_flat and ik_3d.

2. Lesson 7's loop became error, cost, slopes and ik_gd.

3. Check IK with FK every time.

4. raise and try/except handle targets that are out of reach.

5. The Jacobian says how each joint moves the tip, and it makes IK much faster.

Next lesson: AI object detection. The arm finds its own targets.